CASE STUDY DOCUMENT

Full Stack E-Commerce Bookstore Platform

BookNest

Discover. Read. Belong.

Version 1.0 | 2026

1. Synopsis

BookNest is a full-stack e-commerce bookstore platform that enables customers to discover, purchase, and review books online. The system supports two primary roles — Customers and Administrators — each with a dedicated interface. Customers can browse a rich catalog of books by genre, author, title, or keyword; manage a shopping cart and wishlist; place orders via Cash on Delivery or e-wallet payment; track order status; write reviews; and manage their wallet balance.

Administrators maintain the book catalog, manage inventory and stock levels, process and update order statuses, monitor platform analytics, and moderate user reviews. The platform is built on a microservices architecture — modeled directly after the EShoppingZone reference pattern — with independent services for authentication, book catalog, cart, orders, payments/wallet, reviews, notifications, and wishlists, all wired together through a Spring MVC website controller.

The platform also supports GitHub OAuth login, resume/cover-letter style profile management, e-wallet top-up and debit flows with transaction statements, and an in-app + email notification centre for key events such as order placement, payment confirmation, and dispatch updates.

2. Detailed Requirements

2.1 General Requirements

- Users can browse and search books without logging in.
- Users must register or log in (email/password or GitHub OAuth) to purchase, review, or use the wishlist.
- Role-based access control: Customer and Admin roles with distinct permissions.
- All users can update their profiles and manage delivery addresses.
- In-app and email notifications are sent for key transactional events.
- Admin panel provides full catalog, order, user, and analytics management.

2.2 Customer Requirements

- Register, log in, and maintain a personal profile including delivery addresses.
- Browse books category-wise (genre, author, new arrivals, featured).
- Search books by title, author, ISBN, genre, or keyword.
- View detailed book pages with description, specifications, ratings, and reviews.
- Add books to cart, update quantities, remove items, and view cart totals.
- Add books to a wishlist and move items from wishlist to cart.
- Place orders via Cash on Delivery or e-wallet (online payment).
- Track real-time order status: Placed, Confirmed, Dispatched, Delivered, Cancelled.
- View complete order history and download invoices.
- Write, edit, and delete reviews; rate books on a 1–5 star scale.

- Add money to wallet; view current balance and full statement history.
- Receive notifications for order updates, payment confirmations, and alerts.

2.3 Admin Requirements

- Add, edit, update, and delete book listings including cover images and specifications.
- Manage inventory: update stock levels and flag out-of-stock books.
- View and manage all customer orders; update order status at each stage.
- Manage user accounts: view, suspend, or delete customers.
- Access platform analytics: sales totals, top-selling books, category breakdowns.
- View and moderate customer reviews; approve or remove inappropriate content.

2.4 Payment Requirements

- E-wallet supports deposit (top-up) and withdrawal (purchase deduction) transactions.
- Each transaction generates a Statement record with type, amount, timestamp, and remarks.
- Cash on Delivery orders are confirmed without wallet deduction.
- Wallet balance must be validated before online payment is processed.
- Full transaction statement history is accessible from the customer dashboard.

2.5 Notification Requirements

- Email and in-app notifications triggered on: order placement, dispatch, delivery, payment success/failure.
- Admin receives alert notifications for new orders and low-stock books.
- Notification centre displays read/unread state per user with badge count.

3. Use Case Diagram

The diagram below captures all actors and use cases within the BookNest platform, spanning authentication, catalog browsing, cart and wishlist management, checkout and payment, reviews, wallet operations, notifications, and administrative controls.

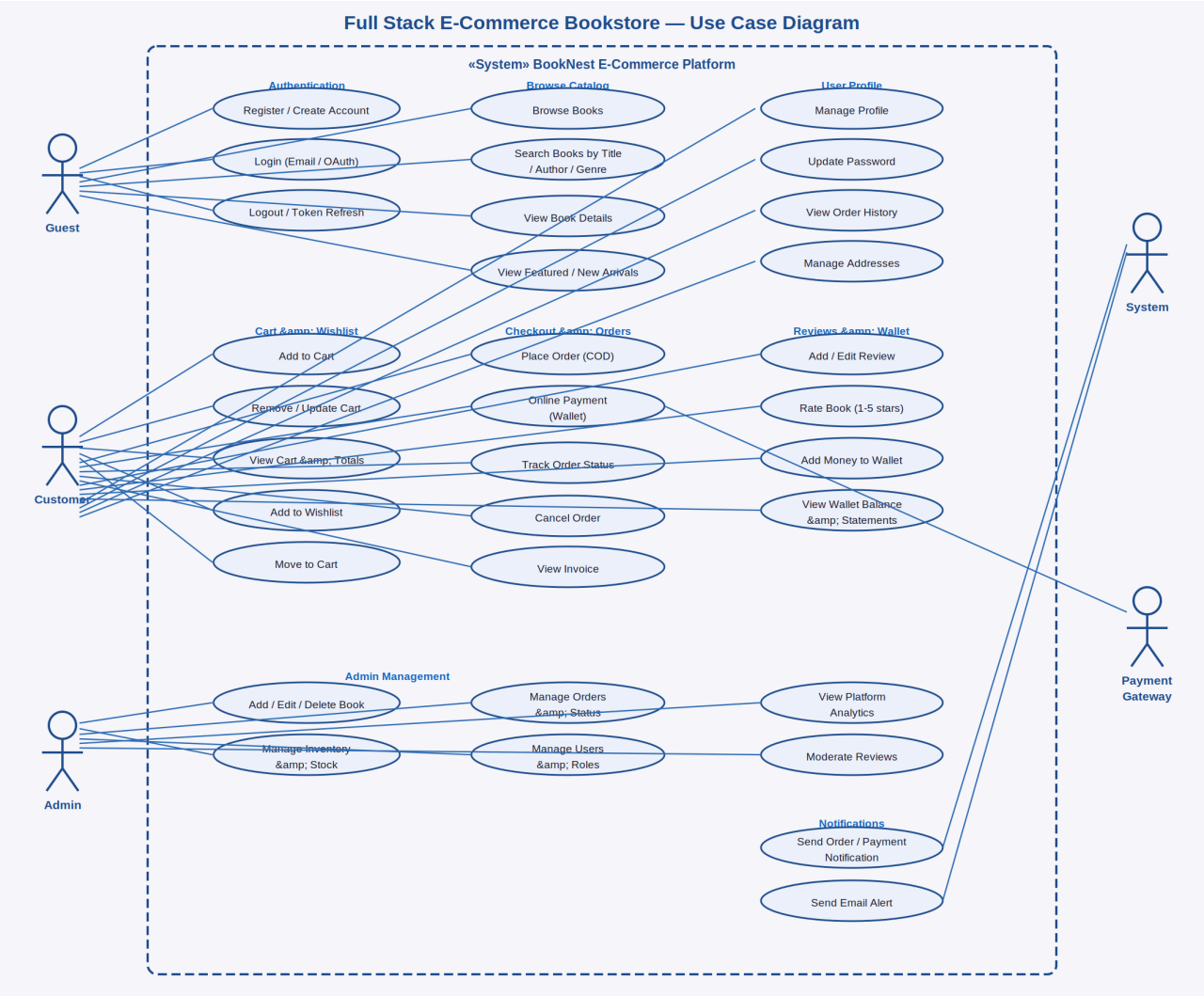

Figure 1: BookNest E-Commerce Bookstore — Complete Use Case Diagram

3.1 Actors

Actor	Description
Guest User	Unauthenticated visitor who can browse catalog and search books
Customer	Registered user who purchases, reviews books and manages wallet/wishlist

Actor	Description
Admin	Platform administrator who manages catalog, orders, users, and analytics
System	Automated component handling notifications, stock updates, and email dispatch
Payment Gateway	External service handling online wallet payment processing

3.2 Use Case Summary

Use Case	Actor(s)	Description
Register / Login	Guest, Customer	Create account or log in via email or GitHub OAuth
Browse Books	Guest, Customer	Browse catalog by genre, author, featured, or new arrivals
Search Books	Guest, Customer	Full-text search by title, author, ISBN, or keyword
View Book Detail	Guest, Customer	View description, specs, price, stock, and reviews
Manage Cart	Customer	Add, remove, update quantity; view cart total
Manage Wishlist	Customer	Add / remove books; move wishlist item to cart
Place Order (COD)	Customer	Checkout with cash on delivery; confirm order
Online Payment	Customer, Payment Gateway	Pay via e-wallet; validate balance and deduct
Track Order	Customer	View real-time order status through delivery pipeline
Write Review	Customer	Rate and review a purchased book (1–5 stars)
Manage Wallet	Customer	Add money; view balance; view statement history
Manage Catalog	Admin	Add, edit, delete books and update inventory stock
Manage Orders	Admin	View all orders; update status at each stage
Manage Users	Admin	View, suspend, or delete customer accounts
View Analytics	Admin	Sales totals, top books, category revenue reports
Moderate Reviews	Admin	Approve or remove customer reviews
Send Notification	System	Dispatch email and in-app alerts for key events

4. Class Diagrams — All Services

BookNest follows the same layered microservices pattern as the EShoppingZone reference: each service contains a POJO/Entity layer, a Repository Interface, a Service Interface, a ServiceImpl class, and a REST Resource class. The nine class diagrams below detail every service's domain model and component relationships.

4.1 Auth / User-Service

The Auth/User-Service handles all authentication and user management concerns: registration, login, logout, JWT token lifecycle, GitHub OAuth integration, profile management, and password changes. It is the security gateway for all protected operations across the platform.

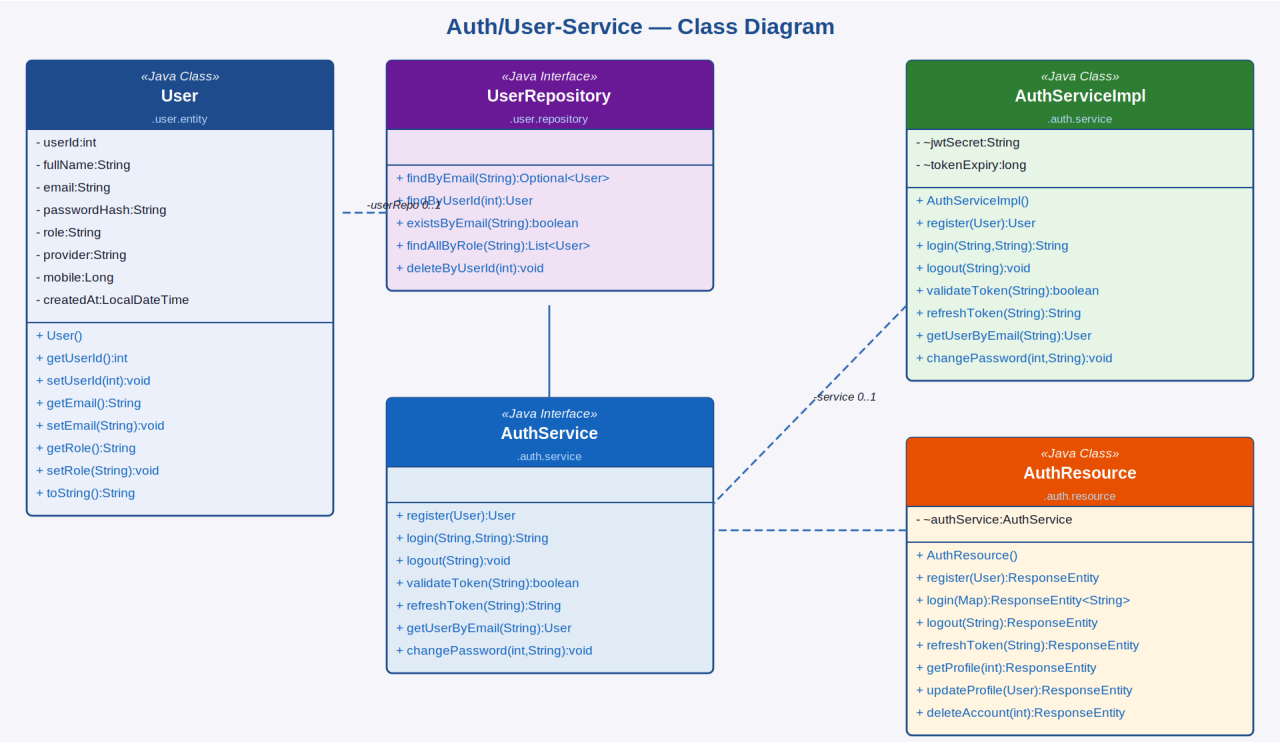

Figure 2: Auth/User-Service — Class Diagram

Component	Type	Responsibility
User	Java Class (Entity)	Stores userId, fullName, email, passwordHash, role, provider, mobile, createdAt
UserRepository	Java Interface	findByEmail(), findByUserId(), existsByEmail(), findAllByRole(), deleteByUserId()
AuthServiceImpl	Java Class	Implements register, login, logout, JWT generation, OAuth handling, password change
AuthService	Java Interface	Declares register(), login(), logout(), validateToken(), refreshToken(), changePassword()
AuthResource	Java Class (REST)	Exposes /auth/register, /auth/login, /auth/logout, /auth/refresh, /auth/profile endpoints

4.2 Book / Catalog-Service

The Book/Catalog-Service manages the entire book inventory: adding new listings, retrieving books by various criteria (genre, author, keyword), updating details, managing stock levels, and curating featured books. It is the primary read-heavy service, serving both guest and authenticated users.

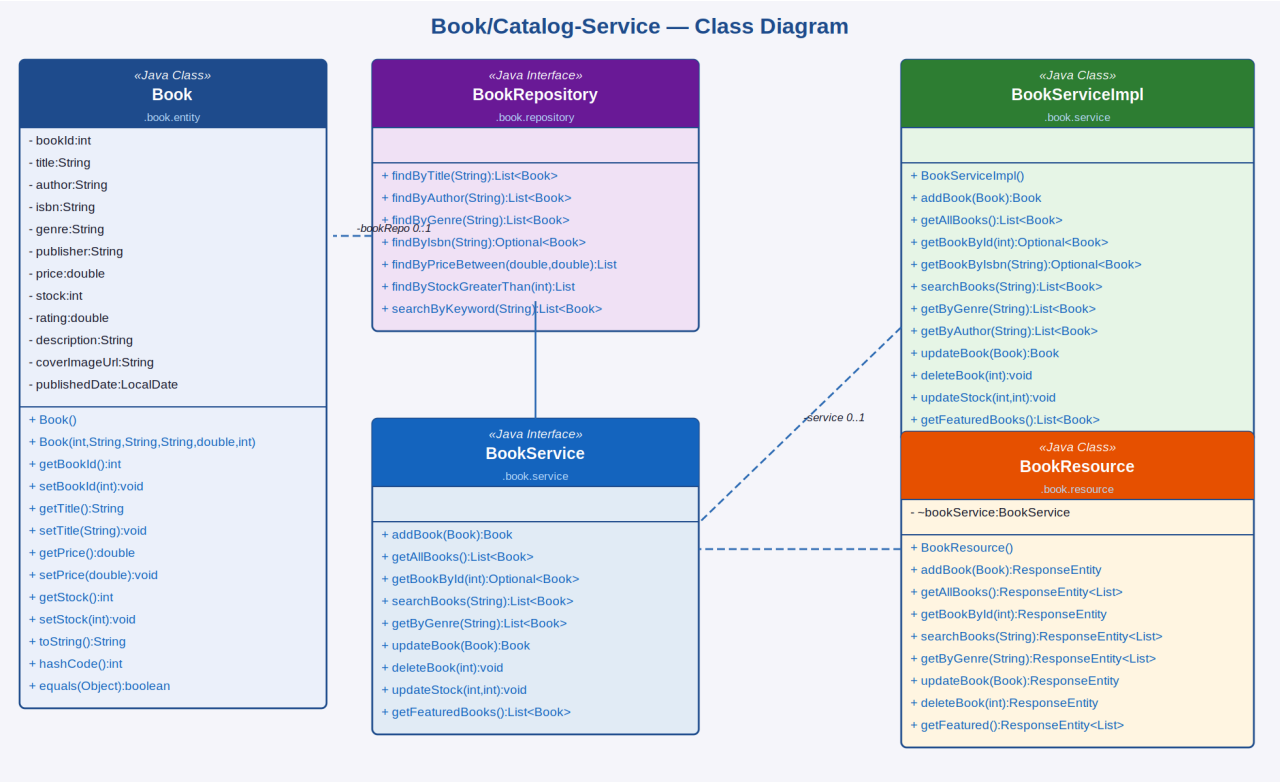

Figure 3: Book/Catalog-Service — Class Diagram

Component	Type	Key Attributes / Methods
Book	Java Class (Entity)	bookId, title, author, isbn, genre, publisher, price, stock, rating, description, coverImageUrl, publishedDate
BookRepository	Java Interface	findByTitle(), findByAuthor(), findByGenre(), findByIsbn(), findByPriceBetween(), searchByKeyword()
BookServiceImpl	Java Class	addBook(), getAllBooks(), getBookById(), searchBooks(), getByGenre(), updateBook(), deleteBook(), updateStock(), getFeaturedBooks()
BookService	Java Interface	Declares all book CRUD, search, filter, and stock operations
BookResource	Java Class (REST)	Exposes /books endpoints: GET (list/search/filter/featured), POST, PUT, DELETE

4.3 Cart-Service

The Cart-Service manages the shopping cart for each logged-in customer. A Cart belongs to one user (userId maps to cartId post-login) and contains a list of CartItems, each capturing the book reference, price, and quantity. The service supports adding, removing, and updating items as well as computing the cart total.

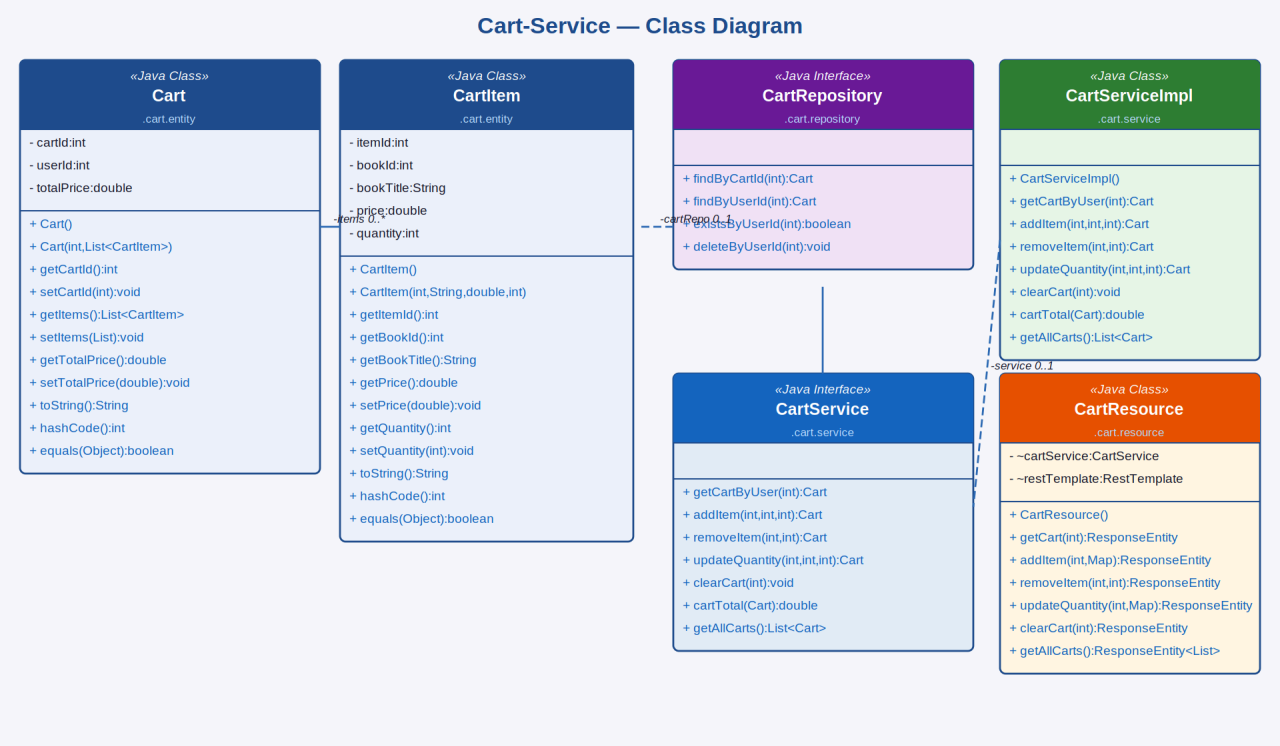

Figure 4: Cart-Service — Class Diagram

Component	Type	Key Attributes / Methods
Cart	Java Class (Entity)	cartId, userId, totalPrice; contains 0..* CartItems
CartItem	Java Class (Entity)	itemId, bookId, bookTitle, price, quantity — linked to Cart
CartRepository	Java Interface	findByCartId(), findByUserId(), existsByUserId(), deleteByUserId()
CartServiceImpl	Java Class	getCartByUser(), addItem(), removeItem(), updateQuantity(), clearCart(), cartTotal(), getAllCarts()
CartService	Java Interface	Declares all cart retrieval and item management operations
CartResource	Java Class (REST)	Exposes /cart endpoints for get, add, remove, update, clear via RestTemplate

4.4 Order-Service

The Order-Service manages the full order lifecycle. It persists Orders linked to an Address and a Book product reference. Two payment flows are supported: Cash on Delivery (placeOrder) and online wallet payment (onlinePayment). Admin can change order status at any stage, and customers can view their full order history.

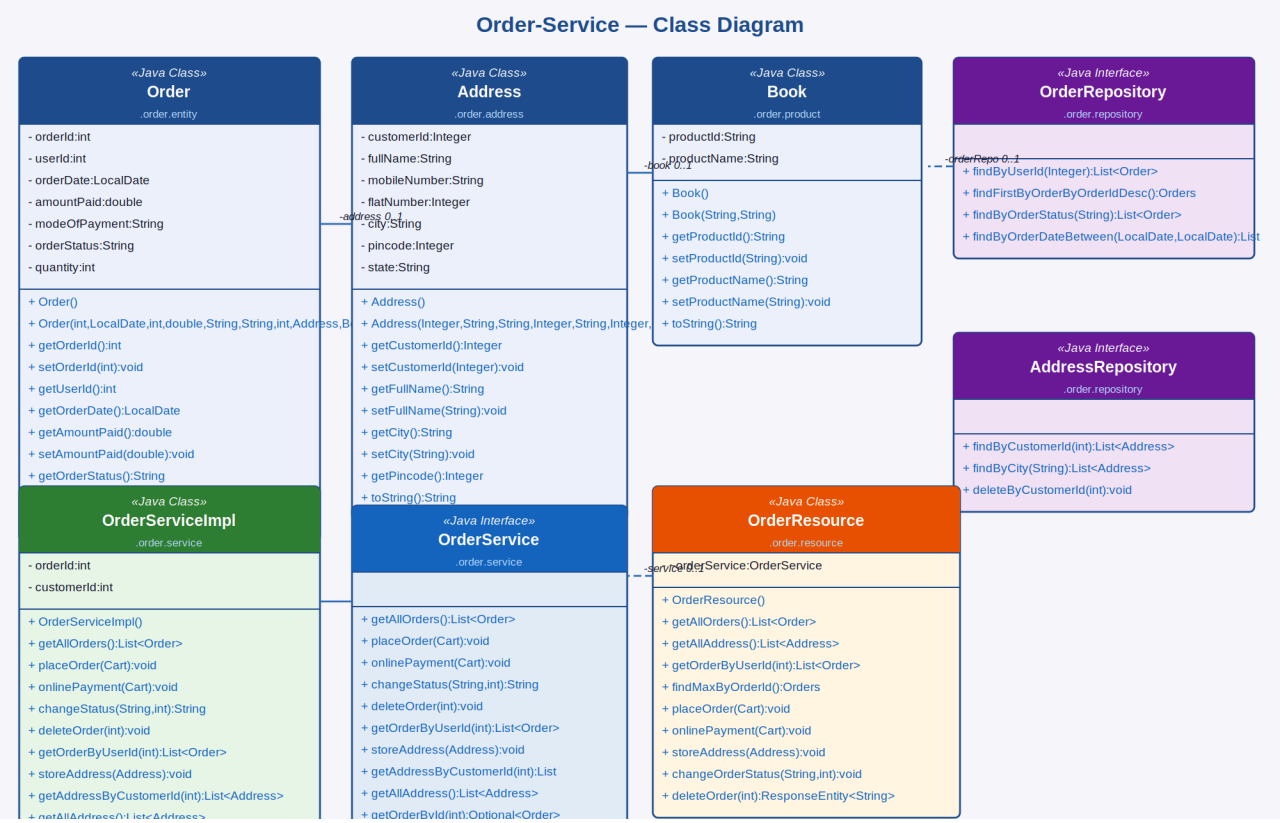

Figure 5: Order-Service — Class Diagram

Component	Type	Key Attributes / Methods
Order	Java Class (Entity)	orderId, userId, orderDate, amountPaid, modeOfPayment, orderStatus, quantity; linked to Address and Book
Address	Java Class (Entity)	customerId, fullName, mobileNumber, flatNumber, city, pincode, state
Book	Java Class (Product ref)	productId, productName — lightweight reference used within order context
OrderRepository	Java Interface	findByUserId(), findFirstByOrderByOrderIdDesc(), findByOrderStatus(), findByOrderDateBetween()
AddressRepository	Java Interface	findByCustomerId(), findByCity(), deleteByCustomerId()

Component	Type	Key Attributes / Methods
OrderServiceImpl	Java Class	getAllOrders(), placeOrder(), onlinePayment(), changeStatus(), deleteOrder(), getOrderByUserId(), storeAddress(), getOrderById()
OrderService	Java Interface	Declares all order placement, retrieval, status management, and address storage operations
OrderResource	Java Class (REST)	Exposes /orders endpoints: GET (all/by user/by id), POST (place/online), PUT (status), DELETE

4.5 Payment / Wallet-Service

The Wallet-Service manages customer e-wallets and their transaction statements. Each Wallet holds a current balance and a list of Statements. The service supports depositing money (addMoney) and debiting for purchases (update/payMoney). All operations generate a Statement record capturing the transaction type, amount, timestamp, and remarks.

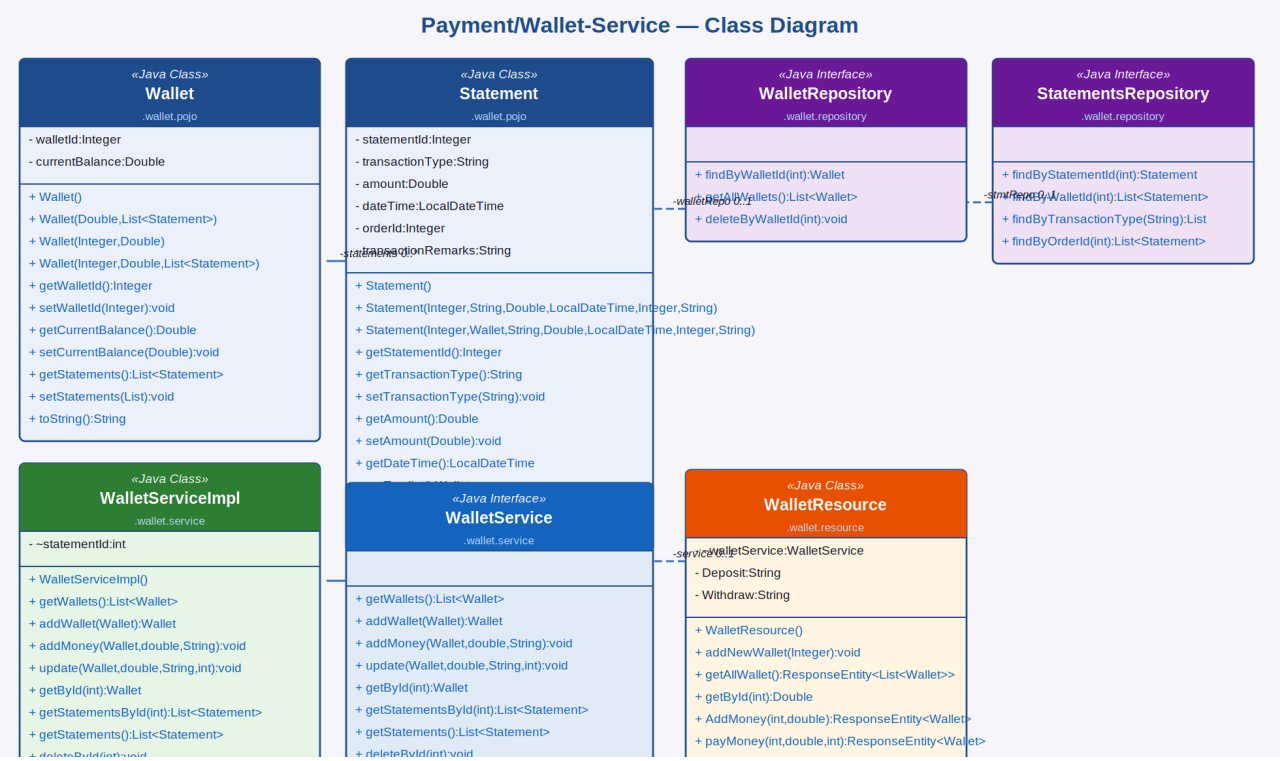

Figure 6: Payment/Wallet-Service — Class Diagram

Component	Type	Key Attributes / Methods
Wallet	Java Class (POJO)	walletId, currentBalance; contains 0..* Statements
Statement	Java Class (POJO)	statementId, transactionType (Deposit/Withdraw), amount, dateTime, orderId, transactionRemarks; linked to Wallet
WalletRepository	Java Interface	findByWalletId(), getAllWallets(), deleteByWalletId()
StatementsRepository	Java Interface	findByStatementId(), findByWalletId(), findByTransactionType(), findByOrderId()
WalletServiceImpl	Java Class	getWallets(), addWallet(), addMoney(), update(), getById(), getStatementsById(), getStatements(), deleteById()
WalletService	Java Interface	Declares all wallet lifecycle and transaction operations

Component	Type	Key Attributes / Methods
WalletResource	Java Class (REST)	Exposes /wallet endpoints: addNewWallet, getAllWallet, getById (balance), AddMoney, payMoney, getStatements, deleteById

4.6 Review-Service

The Review-Service handles all customer reviews and ratings for books. Each review is linked to a specific book and user. The service computes average ratings per book, supports editing and deleting reviews, and provides all reviews for a book or by a user. Only verified purchasers can submit reviews.

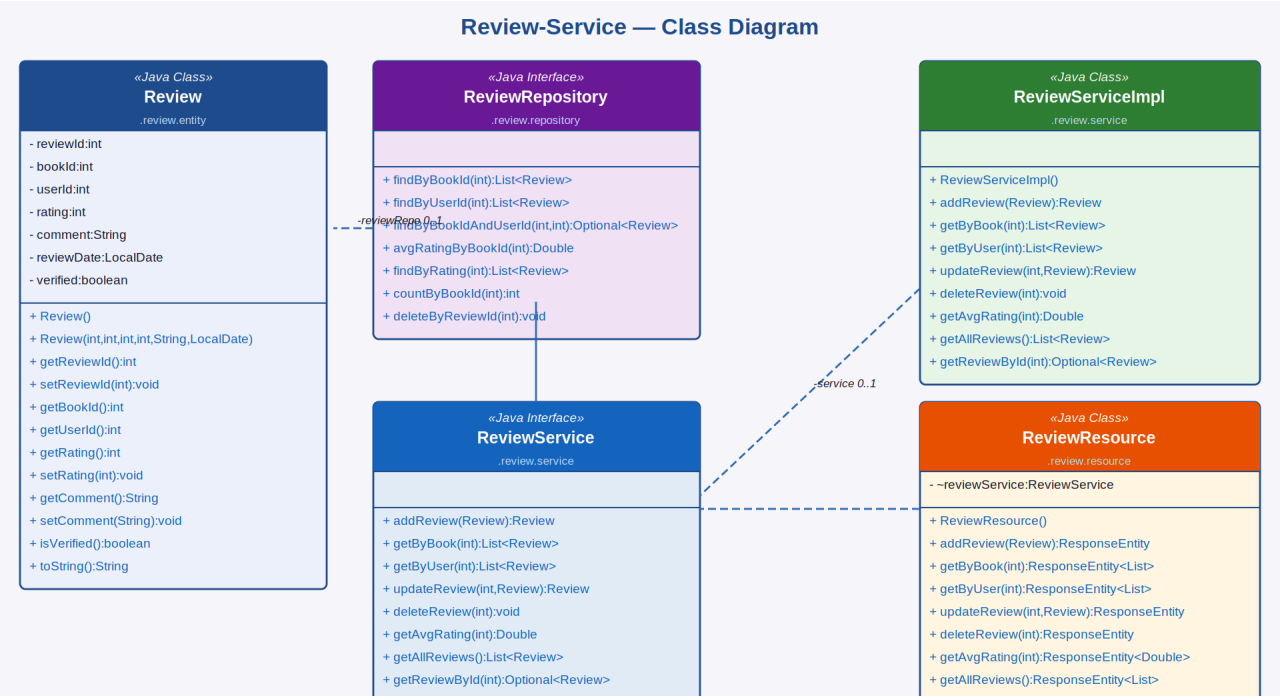

Figure 7: Review-Service — Class Diagram

Component	Type	Key Attributes / Methods
Review	Java Class (Entity)	reviewId, bookId, userId, rating (1-5), comment, reviewDate, verified
ReviewRepository	Java Interface	findByBookId(), findByUserId(), findByBookIdAndUserId(), avgRatingByBookId(), countByBookId(), deleteByReviewId()
ReviewServiceImpl	Java Class	addReview(), getByBook(), getByUser(), updateReview(), deleteReview(), getAvgRating(), getAllReviews(), getReviewById()
ReviewService	Java Interface	Declares all review CRUD and rating aggregation operations
ReviewResource	Java Class (REST)	Exposes /reviews endpoints: POST, GET (by book/user/all), PUT, DELETE, GET avg-rating

4.7 Notification-Service

The Notification-Service dispatches in-app and email alerts for transactional events such as order placement, payment confirmation, dispatch, and delivery updates. It tracks read/unread state per user, supports bulk mark-as-read, and exposes an unread badge count for the UI notification bell.

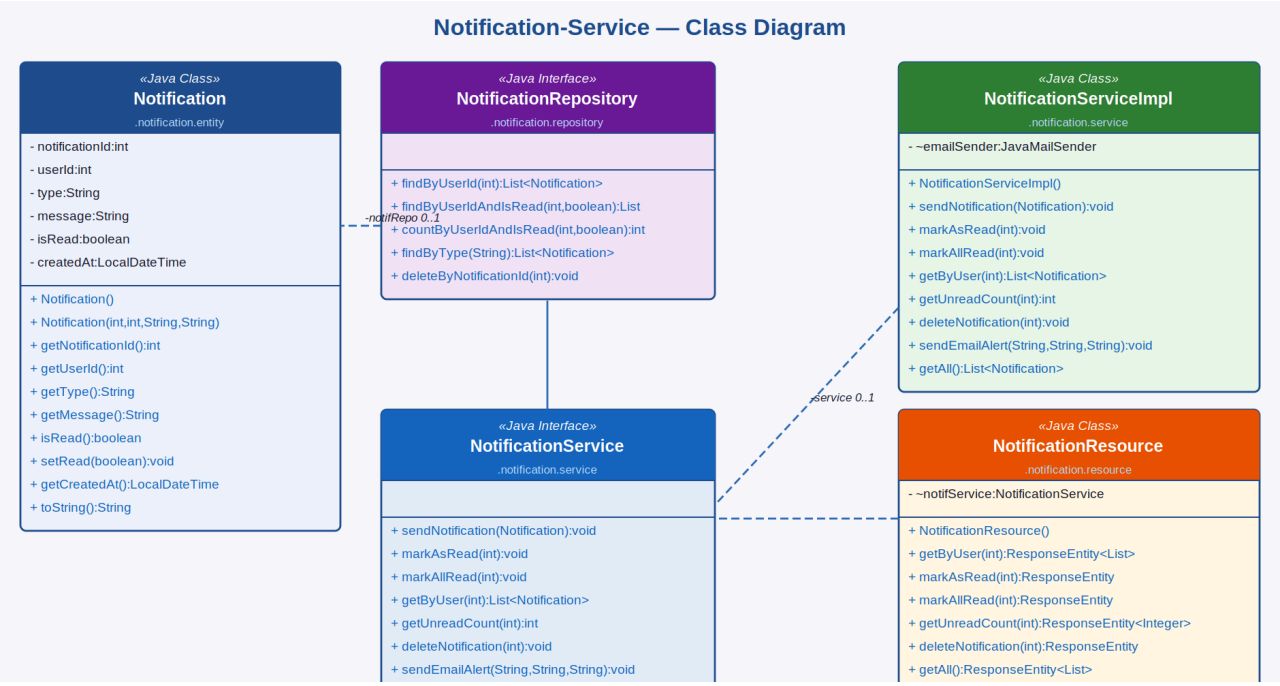

Figure 8: Notification-Service — Class Diagram

Component	Type	Key Attributes / Methods
Notification	Java Class (Entity)	notificationId, userId, type, message, isRead, createdAt
NotificationRepository	Java Interface	findByUserId(), findByUserIdAndIsRead(), countByUserIdAndIsRead(), findByType(), deleteByNotificationId()
NotificationServiceImpl	Java Class	sendNotification(), markAsRead(), markAllRead(), getByUser(), getUnreadCount(), deleteNotification(), sendEmailAlert()
NotificationService	Java Interface	Declares all notification dispatch, retrieval, and read-state management operations
NotificationResource	Java Class (REST)	Exposes /notifications endpoints: getByUser, markAsRead, markAllRead, getUnreadCount, delete, getAll

4.8 Wishlist-Service

The Wishlist-Service allows customers to save books for future purchase. Each Wishlist belongs to one user and contains a list of WishlistItems, each referencing a book with its title and price. Items can be removed individually, the wishlist can be cleared entirely, or individual items can be moved directly to the shopping cart.

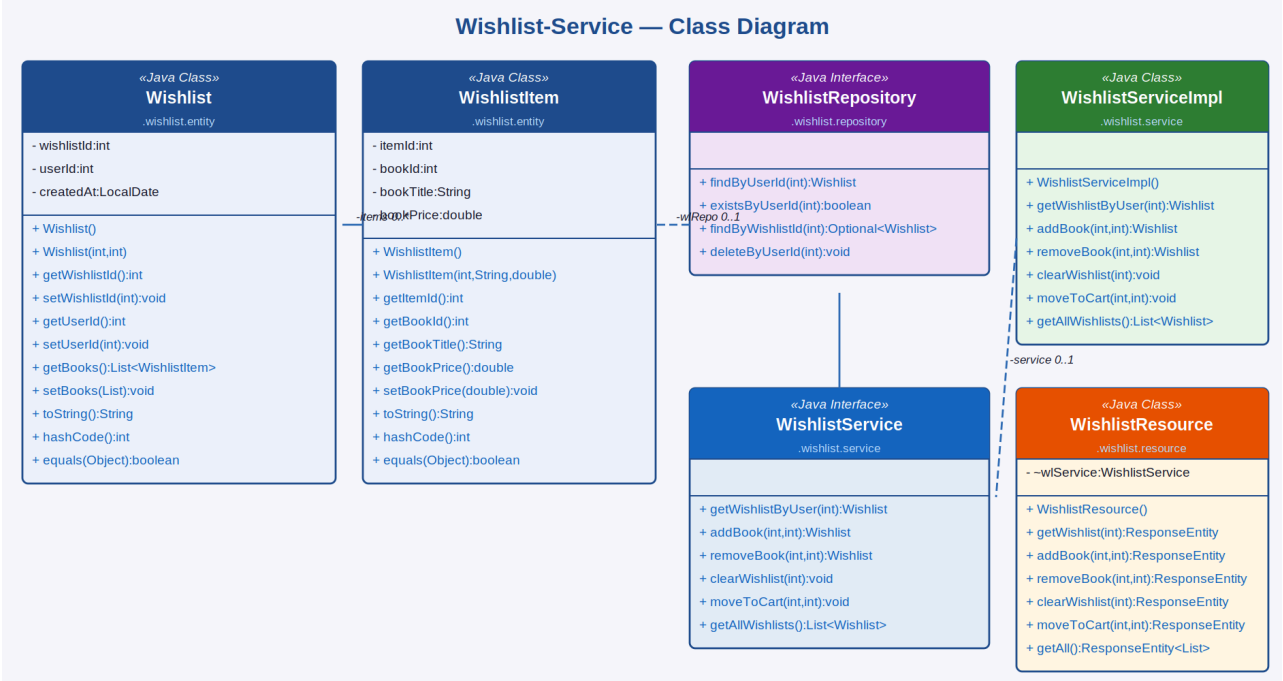

Figure 9: Wishlist-Service — Class Diagram

Component	Type	Key Attributes / Methods
Wishlist	Java Class (Entity)	wishlistId, userId, createdAt; contains 0..* WishlistItems
WishlistItem	Java Class (Entity)	itemId, bookId, bookTitle, bookPrice — linked to Wishlist
WishlistRepository	Java Interface	findByUserId(), existsByUserId(), findByWishlistId(), deleteByUserId()
WishlistServiceImpl	Java Class	getWishlistByUser(), addBook(), removeBook(), clearWishlist(), moveToCart(), getAllWishlists()
WishlistService	Java Interface	Declares all wishlist retrieval, item management, and cart-transfer operations
WishlistResource	Java Class (REST)	Exposes /wishlist endpoints: getWishlist, addBook, removeBook, clearWishlist, moveToCart, getAll

4.9 Website-Controller (MVC Layer)

The Website-Controller is the Spring MVC layer that integrates all underlying microservices and renders the Thymeleaf views for the customer and admin interfaces. It contains three controllers — UserController, BookController, and AdminController — each wiring in the relevant service dependencies via Spring @Autowired injection.

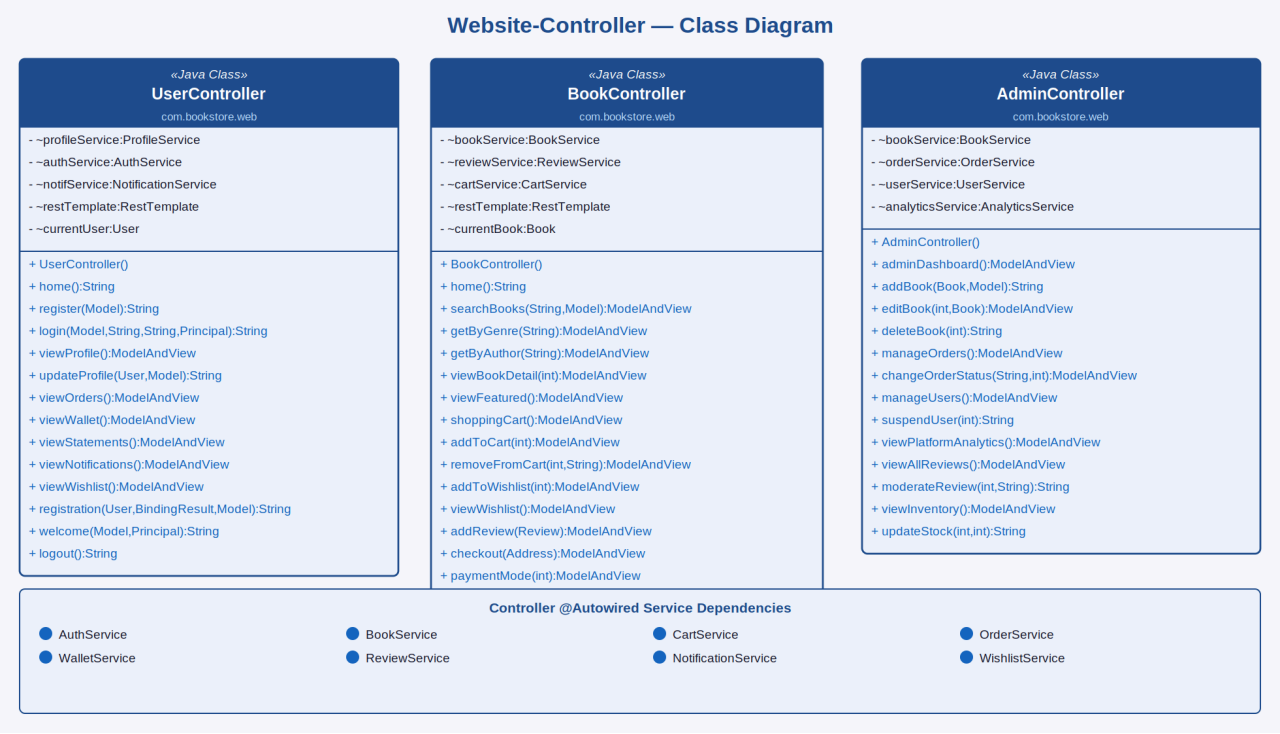

Figure 10: Website-Controller — Class Diagram

Controller	Type	Key Methods
UserController	Java Class (MVC)	home(), register(), login(), viewProfile(), updateProfile(), viewOrders(), viewWallet(), viewStatements(), viewNotifications(), viewWishlist(), welcome(), logout()
BookController	Java Class (MVC)	home(), searchBooks(), getByGenre(), getByAuthor(), viewBookDetail(), viewFeatured(), shoppingCart(), addToCart(), removeFromCart(), addToWishlist(), addReview(), checkout(), paymentMode(), cashOnDelivery(), proceedToPay(), payMoney()
AdminController	Java Class (MVC)	adminDashboard(), addBook(), editBook(), deleteBook(), manageOrders(), changeOrderStatus(), manageUsers(), suspendUser(), viewPlatformAnalytics(),

Controller	Type	Key Methods
		viewAllReviews(), moderateReview(), viewInventory(), updateStock()

5. Microservices Architecture Overview

BookNest is structured as a set of independently deployable microservices following the same pattern as the EShoppingZone reference system. Inter-service communication uses REST via RestTemplate for synchronous calls. Each service owns its own database schema, service interface, implementation, and REST resource layer.

Microservice	Base Package	Primary Domain
auth-service	com.booknest.auth	User registration, login, JWT, GitHub OAuth
book-service	com.booknest.book	Book catalog, search, filtering, stock management
cart-service	com.capgemini.cartservice	Shopping cart, CartItem management, cart totals
order-service	com.cg.order.orderservice	Order placement (COD & online), address, order status
wallet-service	com.capgemini.eshoppingZone.ewallet	E-wallet balance, deposits, withdrawals, statements
review-service	com.booknest.review	Book reviews, ratings, average score aggregation
notification-service	com.booknest.notification	In-app & email notifications, read-state tracking
wishlist-service	com.booknest.wishlist	Customer wishlists, cart transfer
booknest-web	com.capgemini.EshoppingZone.web	Spring MVC controllers, Thymeleaf view rendering

6. Non-Functional Requirements

Category	Requirement
Performance	Book search results must return within 2 seconds for up to 10,000 concurrent users
Scalability	Each microservice is independently scalable via Docker/Kubernetes orchestration
Security	Passwords stored as bcrypt hashes; JWT tokens expire in 24 hours; OAuth2 for GitHub login
Availability	99.9% uptime SLA; health-check endpoints exposed per service; graceful degradation
Data Integrity	ACID-compliant transactions for wallet debit and order placement; no double-spend
Usability	Responsive UI on desktop and mobile; category-wise product display matching EShoppingZone pattern
Maintainability	Services independently deployable; RESTful API contracts versioned under /api/v1/
Auditability	All wallet transactions generate immutable Statement records with timestamps and remarks

7. Proposed Technology Stack

Layer	Technology
Backend Framework	Java Spring Boot (Spring MVC, Spring Security, Spring Data JPA)
Authentication	JWT (JSON Web Tokens), Spring Security OAuth2, GitHub OAuth
Database	MySQL (primary relational store); Redis (session cache)
Search	Elasticsearch or MySQL full-text search for book keyword queries
Frontend	Thymeleaf templates (MVC) integrated with Bootstrap 5 for responsive UI
File / Image Storage	AWS S3 or local file server for book cover image uploads
Payment Integration	Internal e-wallet service; extensible to Razorpay / Stripe for card payments
Messaging (optional)	RabbitMQ for asynchronous notification dispatch on order events
Containerisation	Docker with Docker Compose; Kubernetes for production orchestration
API Documentation	Swagger / OpenAPI 3.0 for all REST endpoints
CI/CD	GitHub Actions or Jenkins pipeline for automated build, test, and deployment

8. Glossary

Term	Definition
Customer	A registered user who browses, purchases, and reviews books on BookNest
Admin	A platform administrator who manages catalog, orders, users, and analytics
COD	Cash on Delivery — order payment method where the customer pays upon receiving the book
E-Wallet	Virtual wallet within the platform used to make online payments for orders
Statement	A transaction record capturing type, amount, timestamp, and remarks for each wallet operation
JWT	JSON Web Token — a compact, URL-safe means of representing authentication claims
OAuth	Open Authorization — open standard for access delegation; used here for GitHub login
Wishlist	A saved list of books a customer intends to purchase in the future
CartItem	A single book entry inside a shopping cart, tracking quantity and price
Microservice	An independently deployable service responsible for a single business domain
Stock	The number of physical copies of a book available for purchase in the inventory
Rating	A 1–5 star score given by a verified purchaser attached to a book review